Chapter 15 — Rapid Review, Future Trends and Glossary

Last reviewed: September 2026. The review sheets summarise this book as written; the trends section describes directions visible in 2026, not predictions of fact.

Everything before this chapter built understanding at chapter depth. An interview — or a design review at work — asks you to pull that understanding out at conversation speed, out loud, under mild pressure. That is a separate skill, and it needs separate practice. This chapter compresses the book into revision tools, then looks outward: where the field is heading, how to keep learning, and every key term in one place.

What you'll learn

- A two-minute review sheet for each of Chapters 1–14, with links to the sections behind every bullet
- Decision cheat sheets: each major either-or with its deciding variable and default
- Twelve "red flag" phrases that make an answer sound junior, and what to say instead
- A 52-question interview bank with compressed answers, plus a script that drills it
- Future trends (model capabilities, agents, AI with IoT/AR/VR/robotics/quantum, the future of work, regulation), stated as trends
- A staged learning roadmap with a project per stage
- An alphabetical glossary of 183 terms, each pointing to the chapter that explains it

How to revise with this chapter

Tool	Rhythm	Why
Review sheets (§15.1)	One chapter a day in the last two weeks	Rebuilds the map of each topic
Decision cheat sheets (§15.2)	Ten minutes daily; cover the right column and reproduce it	The deciding variables must become automatic
Red flags (§15.3)	Read carefully once now, skim the night before	Cheap fixes with the biggest effect on how you come across
Question bank (§15.4)	Shuffled, aloud, ~30 s each, in the final week	Interviews grade spoken recall, not recognition
One full design per session	Pair every drill with one narrated case from Chapter 14 or §13.11	Drills keep vocabulary ready; full designs keep assembly ready

Three drilling rules: **random order, out loud, timed**; after answering, **check the deciding variable** (if you cannot say what drives the choice, reread the section rather than memorising the words); and **re-drill misses after a day or two**, not immediately — answering again straight away only tests short-term memory.

15.1 Rapid-review sheets, chapter by chapter

How to use them

One sheet per chapter: the ideas you must be able to explain without notes, a few numbers worth remembering, and a "can you…?" test. If any bullet feels shaky, follow its section link. Read a sheet in two minutes; explain it aloud in five.

Chapter 1 — The World of AI (Chapter 1)

- **AI** = building agents that act rationally: maximise an expected performance measure given their percepts. Specify tasks with **PEAS** and classify the environment. (§1.1)
- The **Turing test** measures imitation, not understanding. (§1.2)
- **History**: 1956 Dartmouth → symbolic AI → AI winters → statistical ML → deep learning (2012) → transformers (2017) → ChatGPT (Nov 2022) → reasoning models and agents. (§1.3)
- Everything deployed is **narrow** AI; AGI has no agreed definition. (§1.4)

- **Family tree**: AI ⊃ ML ⊃ deep learning ⊃ generative AI ⊃ LLMs; agentic AI wraps models in a plan–act–observe loop; symbolic vs statistical knowledge. (§1.5)
- **Constraints** decide projects: data, cost, latency, reliability compounding over steps (p^n), integration, regulation. (§1.8)
- **Can you…** write PEAS for a self-driving taxi? explain why a 95%-reliable step repeated 10 times is only ~60% reliable?

Chapter 2 — Classical AI and Search (Chapter 2)

- **Formulate** a problem: states, actions, transition model, goal test, path cost. (§2.1)
- **Uninformed**: BFS (shallowest first, memory-hungry), DFS (low memory, no guarantees), iterative deepening, uniform-cost (cheapest path). (§2.2)
- **Informed**: greedy (f = h) is fast but not optimal; **A*** (f = g + h) is optimal with an admissible heuristic. (§2.3)
- **Local search**: hill climbing, random restarts, simulated annealing, genetic algorithms. (§2.4)
- **Games**: minimax, alpha-beta (same answer, far fewer nodes); **MCTS** + learned policy/value networks → AlphaGo. (§2.5, §2.6)
- **CSPs**: backtracking with MRV and forward checking. (§2.7)
- **Can you…** explain why Manhattan distance is admissible on a 4-connected grid? trace one alpha-beta cut-off?

Chapter 3 — Transformers and Embeddings (Chapter 3)

- **Tokens**: subword pieces from BPE-style tokenizers, ~4 English characters each; everything is counted and priced in tokens. (§3.1)
- **Embeddings**: learned vectors where closeness = similar meaning; compare with cosine similarity. (§3.2)
- **Position**: attention is order-blind, so add positional information (sinusoidal, learned, RoPE). (§3.3)
- **Self-attention**: $\text{softmax}(QK^T/\sqrt{d_k})V$; all tokens in parallel; cost grows with T^2. **Multi-head** attention; GQA/MQA share K/V. (§3.4, §3.5)
- **Block**: attention + feed-forward, each with residual and normalisation, stacked; **causal mask** for generation. (§3.6, §3.7)
- **Encoder / decoder / encoder–decoder**: BERT / GPT-Claude-Llama / T5-Whisper. (§3.8)
- **Can you…** compute attention weights for three tokens by hand? say why decoders need the mask?

Chapter 4 — Large Language Models (Chapter 4)

- An LLM is a decoder-only transformer trained to **predict the next token**, generating one token at a time. (§4.1, §4.2)
- **Scaling laws**: training compute $C \approx 6 \cdot N \cdot D$; Chinchilla ≈ 20 tokens per parameter; smaller over-trained models are cheaper to serve. (§4.4)
- **Post-training**: SFT → RLHF or DPO → RL on verifiable rewards for reasoning models. (§4.5, §4.6, §4.11)
- **Decoding**: greedy, beam, temperature, top-k, top-p. (§4.7)
- **Context**: the model is stateless; the window is finite; history must be selected, summarised or retrieved. (§4.8)
- **Inference**: prefill sets time-to-first-token; decode is memory-bound; the KV cache trades memory for speed. (§4.9)
- **Limits**: hallucination, knowledge cutoff, no private knowledge, prompt injection, output variance. **Open vs closed** models. (§4.10, §4.13)
- **Can you…** estimate the training compute of a 7B model on 2T tokens ($\approx 8.4 \times 10^{22}$ FLOPs)? explain why temperature 0 is still not perfectly deterministic?

Chapter 5 — Prompt Engineering (Chapter 5)

- A prompt is a **specification**: role, task, context, constraints, output format, examples; plain direct language, with reasons for rules. (§5.2)
- **Zero-shot vs few-shot**; diverse, tagged examples are the clearest spec. (§5.3)
- **System prompts** for stable rules; **scenario ("act as") prompts** for a stakeholder's viewpoint. (§5.4)
- **Delimit** instructions from data (XML-style tags); **structured output** with schema, validation and retry. (§5.5, §5.6)

- **Reasoning**: chain-of-thought for standard models, effort settings for reasoning models; **self-consistency** votes across samples; **chaining** and **ReAct** split work into checked steps. (§5.7–§5.10)
- **Iterate** one or two changes at a time; **troubleshoot** by finding the missing information; treat outside text as untrusted. (§5.12, §5.13)
- **Evaluate prompts like code**: criteria, test set, graders, versions. (§5.14)
- **Can you…** rewrite a vague one-line prompt into a full specification? name two defences against injected instructions?

Chapter 6 — Building LLM Apps in Python (Chapter 6)

- **API call** = model + system prompt + messages + max_tokens (+ tools); the response has content blocks, a stop reason and token usage; the API is stateless. (§6.2)
- **Streaming** reduces perceived latency. (§6.3)
- **Structured output**: schema-constrained modes or strict tools; still validate. (§6.4)
- **Tool-use loop**: JSON-Schema tools → model asks → your code validates and runs → result back by id → repeat, with limits. (§6.5)
- **Cost** = input tokens × input price + output tokens × output price. (§6.6)
- **Reliability**: backoff + jitter on 429, 5xx and timeouts; no retries on other 4xx. **Caching**: stable prefix first. **Memory**: recent turns + summary + durable facts under a budget. (§6.7–§6.9)
- **MCP**: host / client / server over JSON-RPC; tools (model-controlled), resources (app-controlled), prompts (user-controlled). (§6.10)
- **Can you…** sketch the tool loop from memory? estimate the monthly cost of 10,000 requests a day?

Chapter 7 — Embeddings, Vector Databases and RAG (Chapter 7)

- **RAG** = retrieve evidence, then generate from it; an offline indexing system plus an online request path, evaluated separately. (§7.1)
- **ANN indexes** (HNSW, IVF, PQ) trade recall for speed and memory — measure recall against exact search. (§7.3)
- A **vector database** adds filtering during search, updates/deletes and replication to an index. (§7.4)
- **Ingestion** decides quality: parsing, structure-aware chunks with overlap, metadata, sync. (§7.5)
- **Request path**: process the query (rewrite, HyDE, routing), retrieve, assemble a budgeted context, generate, then run post-checks with a fallback. (§7.6)
- **Hybrid** BM25 + vectors fused with RRF (Σ 1/(60 + rank)); cross-encoder reranking. (§7.7)
- **Grounding, permissions inside retrieval (fail closed)**, freshness and deletion. (§7.8, §7.9)
- **Evaluate**: recall@k, faithfulness, answer relevance, refusal correctness, zero leaks. (§7.11)
- **Can you…** size raw vectors for 10M chunks × 1,024 float32 dims (≈ 41 GB)? say which stage failed when the right document exists but is never retrieved?

Chapter 8 — Prompting vs Fine-Tuning vs RAG, and LLM Evaluation (Chapter 8)

- **Adaptation**: prompting is the baseline; **RAG supplies knowledge** (fresh, private, permissioned, citable); **fine-tuning changes behaviour** (format, tone, narrow skill, cheaper small models). Diagnose the failure first; never use weights as access control. (§8.1)
- **LoRA** trains two small matrices ($h = W_0x + (\alpha/r)\cdot BAx$): about 65K trainable parameters instead of 16.8M for one 4096 × 4096 matrix at r = 8. **QLoRA** adds a 4-bit base model. Full fine-tuning of a 7B model needs roughly 16 bytes per parameter. (§8.2)
- **Tuning data**: instruction pairs (SFT), preference pairs (RLHF/DPO), teacher outputs (distillation); re-run every eval after tuning — forgetting and safety erosion happen. (§8.3)
- **Evaluation pyramid**: programmatic checks → LLM-as-judge (watch position, verbosity and self-preference bias) → human review, over a versioned golden set run like CI. (§8.4, §8.5)
- **Metrics**: exact match/F1, pass@k for code, BLEU/ROUGE only as rough signals, faithfulness/relevance/refusal for RAG — report uncertainty. (§8.6)
- **Online evaluation** (A/B, canaries, outcome metrics) and **guardrail suites** as hard gates. (§8.7, §8.8)
- **Can you…** pick RAG, prompting or tuning for three different failures? compute LoRA's trainable parameters for one matrix?

Chapter 9 — AI Agents (Chapter 9)

- **Agent** = LLM + tools + memory + a loop in which the model picks the next step at run time; if you can draw the flowchart, build a **workflow** instead. (§9.1)
- **Loop / ReAct**: reason → act → observe, bounded by steps, tokens, money, time and actions; errors compound ($0.95^{10} \approx 0.60$), so verify intermediate results. (§9.2)
- **Tool calling**: the model *requests*; the runtime validates, authorises, executes and logs. Few, coarse, well-described tools; idempotent writes. (§9.3)
- **Guardrails**: least privilege, user-scoped credentials, consequence tiers, an action gate, targeted human approval — injection is stopped by the gate, not the prompt. (§9.4)
- **Planning** (plan-and-execute, reflection) and **memory** (window, compaction, long-term, durable task state). (§9.5, §9.6)
- **Multi-agent**: default to one; split only for isolation, parallelism or specialisation. **MCP** standardises the plug but does not replace authorisation. (§9.7, §9.8)
- **Evaluate** whole trajectories in a sandbox (success with cost, pass^k); **control** with a four-rung autonomy ladder per action tier, budgets, kill switches and alarms. (§9.9, §9.10)
- **Can you…** design the customer-support agent's tools, tiers and approval gates (§9.11)? explain pass^k?

Chapter 10 — Generative Models Beyond Text (Chapter 10)

- **Families**: VAEs (smooth latent space, blurrier), GANs (sharp and fast, mode collapse), autoregressive (token by token), diffusion/flow (top quality, slow without tricks). (§10.1)
- **VAE**: encode to a distribution, sample $z = \mu + \sigma\varepsilon$, loss = reconstruction + KL; it is the compressor inside latent diffusion. (§10.2)
- **GAN**: generator vs discriminator game. (§10.3)
- **Diffusion**: add noise on a schedule, train a network to predict the noise, generate by denoising step by step (U-Net or diffusion transformer). (§10.4)
- **Text-to-image**: CLIP aligns text and images; cross-attention injects the prompt; classifier-free guidance trades diversity for prompt adherence; Stable Diffusion = VAE + text encoder + denoiser in latent space. (§10.5)
- **Audio** (ASR with Whisper, TTS, voice cloning), **video** (temporal consistency is the hard part), **multimodal** models. (§10.6–§10.8)
- **Evaluate** with FID, precision/recall, CLIP score and human comparisons; **risks**: deepfakes, consent, copyright; watermarks and C2PA provenance. (§10.9, §10.10)
- **Can you…** explain classifier-free guidance in two sentences? say why diffusion runs in latent space?

Chapter 11 — LLM Serving and Inference (Chapter 11)

- Treat the LLM as a **production component** with latency, cost and failure modes. (§11.1)
- **Prefill** (parallel, compute-bound) sets time-to-first-token; **decode** (sequential, memory-bandwidth-bound) sets tokens per second; end-to-end ≈ TTFT + TPOT × (n − 1). (§11.2, §11.3)
- **KV cache** per token = 2 × layers × KV-heads × head-dim × bytes — usually the real concurrency limit. (§11.4)
- **Continuous batching** and **PagedAttention** (vLLM) keep GPUs busy and waste less memory. (§11.5, §11.6)
- Make models cheaper: **quantization**, **distillation**, **speculative decoding** (same output distribution, fewer large-model passes). (§11.7–§11.9)
- **Memory sizing** = weights + users × context × KV-per-token + overhead; tensor parallelism inside a node, pipeline across nodes. (§11.10, §11.11)
- **Avoid calls**: exact, semantic and prefix caching; **routing/cascades** to cheaper models. (§11.13, §11.14)
- **Cost**: per unit → price → volume → decision; API vs fleet; operate with rate limits, backpressure, fallbacks and observability. (§11.15–§11.17)
- **Can you…** size memory for a 70B model at 16-bit (weights alone ≈ 140 GB)? price the inference layer for a large user base out loud (§11.18)?

Chapter 12 — Safety, Ethics and Governance (Chapter 12)

- **Root cause**: instructions and data share one channel; the model is never the security boundary — enforce what matters in code. (§12.1)

- **Hallucination**: ground with retrieval and citations, allow abstention, keep humans on high-stakes outputs. (§12.2)
- **Bias** comes from history, representation, proxies and associations; measure per group; dropping the sensitive column is not enough. (§12.3)
- **Prompt injection**: direct vs indirect (hidden in content the model reads); defend with screening, data tagging, least privilege and **action gates**. (§12.4)
- **Privacy**: consent, GDPR/CCPA rights, PII redaction, per-user scoping of retrieval and caches. (§12.5)
- **Guardrails**: input gate → hardened assembly → output gate → action gate; measure over-refusal too. (§12.6)
- **Misuse** and unreliable AI-text detection; **alignment** (SFT, RLHF, Constitutional AI, DPO) makes good behaviour likely, not guaranteed; continuous **red-teaming**. (§12.7–§12.9)
- **Copyright, environmental cost, governance**: EU AI Act risk tiers (high-risk duties now from 2 Dec 2027 for Annex III uses, 2 Aug 2028 for Annex I products), NIST AI RMF (Govern, Map, Measure, Manage); a deployment checklist. (§12.10–§12.13)
- **Can you…** explain indirect prompt injection with an email-summarising assistant? list the four gates?

Chapter 13 — AI System Design and GenAI Products (Chapter 13)

- Identify the **interview form** and confirm the depth wanted in the first minute. (§13.1)
- **What changes when the system learns**: quality becomes a metric, a feedback loop is required, cost and latency budgets matter, and failures are silent. (§13.2)
- **Three families**: prediction (funnel), content (context + cost + evaluation + safety), action (tools + permissions + limits). (§13.3)
- **Seven-step framework** with a time budget: frame → success metrics → data → model → serving → keep it working → failures and cost. (§13.4)
- **Requirements as numbers**: TTFT/p95, quality metric, cost per unit, safety rules, scale, privacy. (§13.5)
- **Build vs buy**: API → RAG for facts → fine-tune for behaviour → self-host for control or high steady volume. (§13.6)
- **Reference architecture**: gateway → input guard → cache → context assembly (prompt registry, memory, permission-filtered RAG) → router → model → output guard → action gate, with traces feeding evaluation. (§13.7)
- **LLMOps**: version everything, golden sets, regression tests, shadow → canary → A/B. **Cost modelling** and scale estimates. (§13.8, §13.9)
- **Archetypes**: chat, code, search, image, SaaS platform. (§13.10, §13.11)
- **Can you…** draw the reference architecture from memory and put a latency budget on each arrow? explain why chat history cost grows quadratically with turns?

Chapter 14 — AI Case Studies (Chapter 14)

- **Running a case**: a time budget per phase, an exit sentence for each, timeboxing aloud; handle redirects, stress tests and disagreement by naming the deciding variable. (§14.1)
- **Legal research assistant**: wrong citations are the metric that binds; per-claim verification before release; refusal is a product feature; fail closed. (§14.2)
- **Meeting assistant**: a workflow, not an agent; bursty load behind a queue; access rights bound what can be said; deletion propagates to summaries. (§14.3)
- **Text-to-image at scale**: do the maths first (requests/s × seconds per image = busy GPUs); queues drive pricing, autoscaling and UX; moderation gates on prompt and image. (§14.4)
- **Coding agent**: tests inside the loop, CI outside, humans at the end; sandbox without credentials; treat the repository as untrusted input. (§14.5)
- **Critiquing a design**: invariants → loops → reliability → economics → quality, each finding with consequence and fix. (§14.6, §14.7)
- **Email sentence completion**: small distilled LM under a ~100 ms keystroke budget; avoid requests on the client; beam search + confidence threshold tuned on trigger vs acceptance; never guess people's attributes. (§14.8)
- **Machine translation**: one multilingual encoder-decoder with language tags instead of 9,900 pair models; back-translation for low-resource languages; per-pair chrF/COMET and human MQM, never BLEU alone. (§14.9)
- **Image captioning**: image encoder + bridge + LM, run async at upload; grounding rerank and CHAIR against hallucinated objects; attribute policy outside the model. (§14.10)

- **GAN faces**: StyleGAN mapping network and styles; judge by coverage, FID and precision/recall, not losses; truncation trades diversity for quality; a near-duplicate filter makes "not a real person" measurable. (§14.11)
- **High-resolution synthesis**: compress first (latent f = 8), generate at the trained resolution, upscale only kept images; 2× side = 4× tokens = 16× attention. (§14.12)
- **Personalised headshots**: per-user LoRA with a rare token and prior preservation (or a zero-shot identity encoder); photo QC and same-person check before GPU spend; biometric consent and verified deletion. (§14.13)
- **Text-to-video**: spacetime latents, factorised attention, frames denoised jointly for consistency; recaptioned training video; async multi-GPU jobs with previews and checkpoints. (§14.14)
- **Can you…** narrate one of these cases in 45 minutes without notes?

15.2 Decision cheat sheets

What it is

Most AI design questions come down to a handful of either-or choices. Each sheet below gives the **options**, the **deciding variable** (the fact about the requirements that picks the option), and a **default** with its main exception. In an interview, the sentence shape to practise is: *"The deciding variable here is X; ours is Y, so I'd choose Z, which costs us W."*

Defaults are starting points. Several choices depend on more than one requirement, and the exceptions matter as much as the defaults.

Which kind of system is this?

What the model outputs	Family	The design centres on	Where in this book
Scores or labels about items or events	**Predictive**	Features, candidate funnel, metrics, feedback loop	ML Handbook ch. 15–18
New content: text, code, images	**Generative**	Retrieval, cost per request, evaluation, safety	Chapters 4–8, 10, 13
Tool calls that change things in the world	**Agentic**	What it may touch, loop limits, who signs off	Chapter 9

Many real products mix families (a support assistant retrieves, generates *and* acts); name the dominant one and design its concerns first.

Workflow or agent?

- **Deciding variable:** can you draw the flowchart at design time?
- **Default:** yes → a fixed **workflow** (cheaper, testable, readable). Use an **agent** loop only for the part whose path depends on what is discovered at runtime, and bound it (steps, cost, time, which actions).

Build a model or call an API?

- **Deciding variable:** does quality come from your own proprietary data (your users' clicks, purchases, outcomes)?
- **Predictive** systems usually train on that data (bought or managed models can still fit). **Generative** features usually start on a hosted API; privacy, licensing, availability or scale can later favour self-hosting. Revisit with measured costs.

Prompt, RAG or fine-tune?

What is failing	Reach for	Why
Missing, stale, private or per-user facts	**RAG**	Sources can be updated, cited and permission-filtered
Wrong format, tone, length, or a narrow skill	**Prompting first**, then **fine-tuning**	Behaviour is learned into the parameters, so you need training examples and a test set
Answers are good but the bill is too high at scale	A **smaller fine-tuned model**, with a router sending it the easy traffic	The problem is cost, not what the model can do

Tuning can inject knowledge too, but it is a poor substitute for retrieval when facts change or must be traceable. Training from scratch needs an exceptional reason and budget.

Batch, online or streaming inference?

Situation	Mode
Inputs known in advance; slightly stale results are fine	**Batch** – precompute into a key-value store
Inputs only exist at request time	**Online** – with a latency budget and fallbacks
Events, not users, trigger the work	**Streaming** – queue-fronted scoring
Huge catalogue plus need for fresh context	**Hybrid** – precomputed candidates, light online re-ranking

The serving-optimisation order

1. **Avoid work** – safe caching, routing easy requests to smaller models, sending less context.
2. **Execute faster** – batching, KV-cache management, prefix caching, speculative decoding.
3. **Shrink the model** – quantisation, distillation (check quality!).

Measure the bottleneck first; every optimisation carries its own cost (cache invalidation, stale answers, routing mistakes).

Hosted API or self-host?

- **Deciding variable:** sustained token volume versus the cost of a GPU fleet at the utilisation you can realistically achieve, gated by governance needs and team skills.
- **Default:** API first; mixed setups (different tiers per task, overflow capacity) are common.

Synchronous or asynchronous serving?

- **Deciding variable:** generation time versus user patience.
- A few seconds of text → synchronous, streamed. Long jobs (images, video, long documents) → a persistent **job queue**, with priority levels and progress shown to the user.

Vector, keyword or hybrid retrieval?

- **Deciding variable:** the shape of the queries. IDs, codes and exact phrases favour **keyword** (lexical, e.g. BM25); paraphrases and concepts favour **vector** (dense) search.
- **Default: hybrid**, merged with **reciprocal rank fusion**, plus a **cross-encoder reranker** when the top results are not precise enough. Choose a vector store by benchmarking your corpus, filters, traffic and update rate – not by brand.

Where do permissions live?

The one sheet with no trade-off: **below the model**. Enforce access in the retrieval query and in tool services, check current permissions before content reaches the model, and **fail closed** (if the check cannot run, deny). Never "in the prompt", never after generation.

How much grounding?

- **Deciding variable:** stakes.
- Low stakes → context-only instructions, citations that must resolve to real sources, sampled quality review. High stakes → mandatory checks before release, refuse or escalate when support is weak, expert review where needed.

Which evaluation layer?

- **Can code assert it?** → programmatic checks (format, schema, exact values, citations resolve).
- **Needs judgement at scale?** → an **LLM judge** with a written rubric, calibrated against human scores.
- **High stakes or new kinds of failure?** → human experts.
- Offline results must be confirmed online; engagement gains never override safety or correctness.

How does anything ship?

Offline eval against the current version → **shadow** (the new version runs silently alongside) → **canary** or **A/B test** on a small, fixed ("sticky") slice with a named rollback metric → ramp up. Agents also climb a four-rung **autonomy ladder**, separately for each action tier — proposals only, then human-approved actions, then bounded autonomy, then audited autonomy — and drop back a rung automatically on regression.

How often to retrain?

- **Deciding variable:** how fast quality actually decays (measured), plus adversarial pressure (fraud, spam).
- A drift alarm should trigger an **investigation**, not automatically a retrain. Online learning is possible for some workloads but needs validation, rollback and protection against poisoned feedback.

The one-table interview

Moment	Move
Question lands	Classify the family; name a known system it resembles
~Minute 2	Ask which depth they want: modelling or system
~Minute 8	Success metrics **with numbers**; say what is out of scope
Middle	Funnel, pipeline or loop skeleton, with a latency/cost budget on each arrow
Every choice	Deciding variable first, then its cost
~Minute 38	Close the loop: logs → evals → retrain → safe rollout
Closing	Top failure modes with their alarms; unit cost × volume

Rehearsal: cover the right-hand column and reproduce it from the left.

15.3 Red flags that sound junior (and what to say instead)

Interviewers listen for signs that you could actually operate what you propose. These twelve phrases restate the book's lessons in negative form.

Three that can sink an interview

Red flag	Why it hurts	Say instead
"Let's have an LLM order the feed."	An off-the-shelf LLM has never seen your users' clicks; calling it per candidate is too slow and costly at feed scale	"Ranking is a model trained on our interaction logs; LLMs can help offline, e.g. enriching item features or generating labels."
"Step one: we pretrain our own model on everything we have."	Pretraining a large model is a huge, rarely justified investment; even fine-tuning needs a named failure	"Start by prompting a hosted model; bring in retrieval when knowledge is missing; only then consider tuning a compact model, if behaviour stops improving or traffic makes it pay."
An agent loop with no limits and admin credentials	Uncontrolled cost and damage	"Caps on steps, spend, time and actions; tools tiered by risk; approval gates on irreversible actions; autonomy earned step by step."

The three you will hear most often (all easy to repair)

Red flag	Say instead
Requirements without numbers ("fast and accurate")	"Resolve >50% of tickets; first token < 1 s, streamed; cost per conversation < 10% of a human-handled one."
The diagram stops at the response arrow	Log predictions and outcomes, evaluate, retrain, roll out safely — draw it and say how often each step runs
Model calls treated as free	Estimate cost per request × volume out loud; then avoid calls → speed them up → shrink the model

The control flags — a control the model can bypass

Red flag	Say instead
"We'll fine-tune the hallucinations away."	Retrieval for current or private facts, citations that must resolve, per-claim checks, and refusal as a designed outcome
"Permissions are in the prompt."	Access filters in the retrieval query, user-scoped tool credentials, action gates — the model never decides authorisation
"The model will just know" / no evaluation plan	"Every change runs against a golden test set, like code through CI; these online metrics decide promotion."

The remaining three

Red flag	Say instead
Multi-agent by default	One agent by default; add more only for context isolation, parallel independent subtasks, or specialised tools/models — and say which
Brand names as architecture ("Pinecone, LangChain, GPT")	Describe the component and its deciding variable, then a product as one example; benchmark what you already run first
Assuming failures raise errors	Wrong answers return HTTP 200. Name the silent failure, how far it spreads before someone notices, and the alarm (drift monitors, groundedness sampling, action-rate anomalies)

Meta-flags across the whole hour

1. **Depth only when asked** — pair every choice with its cost unprompted ("batch: degrades gracefully, but results are stale").
2. **Treating follow-ups as attacks** — "what if the provider is down?" is an invitation to describe the fallback, its limits and its alarm.

3. **The invisible user** – say what the person experiences during waits, errors and refusals: progress messages, partial results, a way to recover.

Self-audit drill: record one mock interview and count flags in the second group – numbers, the loop, the bill.

15.4 Interview question bank

How to use it

Each answer is a **compressed recall anchor**, not a full explanation – the linked section does the teaching. Drill using the three rules above (`code/drill.py` shuffles the bank for you), and count an answer as correct only when the **deciding variable** or core mechanism came out without peeking.

Foundations (Chapters 1–3)

Q1. What is a rational agent, and how do you specify its task? A. An agent that picks the action expected to maximise its performance measure, given what it has perceived. Specify the task with PEAS – Performance measure, Environment, Actuators, Sensors – then classify the environment (observable? deterministic? static?). *(→ §1.1)*

Q2. How do AI, ML, deep learning, generative AI and LLMs relate? A. Nested sets: AI ⊃ machine learning (learns from data) ⊃ deep learning (many-layer neural networks) ⊃ generative AI (creates content) ⊃ LLMs (generate text). Agentic AI wraps models in a plan–act–observe loop. *(→ §1.5)*

Q3. BFS, DFS or A* – which and why? A. BFS finds the fewest-steps path but stores the whole frontier ($O(b^d)$ memory); DFS uses little memory but gives no optimality guarantee; A* expands by $f = g + h$ and is optimal when the heuristic never overestimates (admissible) – the cost is memory. *(→ §2.2, §2.3)*

Q4. What does alpha-beta pruning change about minimax? A. Nothing about the answer – it skips branches that cannot affect the final decision, so it searches far fewer nodes (best case roughly the square root of minimax's count with good move ordering). *(→ §2.5)*

Q5. How did AlphaGo combine search and learning? A. Monte Carlo tree search guided by a policy network (which moves to try) and a value network (how good a position is). The networks were first trained on human expert games, then improved by self-play reinforcement learning; AlphaGo Zero later dropped the human data. Search improves play, and play generates training data. *(→ §2.6)*

Q6. What is a token, and why should an engineer care? A. A subword piece produced by a tokenizer such as BPE – roughly four characters of English. Context limits, latency and price are all counted in tokens, and odd behaviour (spelling, arithmetic, non-English cost) often traces back to tokenization. *(→ §3.1)*

Q7. What is an embedding and how are two compared? A. A dense vector learned so that similar meanings land close together; compare with cosine similarity (the angle between vectors). *(→ §3.2)*

Q8. Write the attention formula and explain the √d_k. A. $\text{Attention}(Q, K, V) = \text{softmax}(QK^T/\sqrt{d_k})\cdot V$. Each token's query is scored against every key; the softmax weights mix the values. Dividing by $\sqrt{d_k}$ keeps dot products from growing with dimension, which would saturate the softmax and shrink gradients. *(→ §3.4)*

Q9. Encoder, decoder, or encoder–decoder? A. Encoders (BERT) read the whole input bidirectionally – good for understanding and embeddings. Decoders (GPT, Claude, Llama) use a causal mask and generate left to right. Encoder–decoders (T5, Whisper) transform one sequence into another. *(→ §3.8)*

Large language models (Chapter 4)

Q10. How is an LLM trained, end to end? A. Pretraining on next-token prediction over trillions of tokens → supervised fine-tuning on instruction/response examples → preference tuning (RLHF with a reward model, or DPO) → often reinforcement learning on verifiable rewards for reasoning. *(→ §4.3, §4.5, §4.6)*

Q11. What did the Chinchilla scaling result say? A. For a fixed compute budget, train on about 20 tokens per parameter; many earlier models were too big for their data. In practice teams now over-train smaller models because they are cheaper to serve. *(→ §4.4)*

Q12. What do temperature and top-p do? A. Temperature divides the logits before softmax (low = sharper, more predictable; high = more varied). Top-p samples only from the smallest set of tokens whose probabilities sum

to p. Neither guarantees determinism, and some reasoning models reject these settings and are steered with an effort setting instead. *(→ §4.7)*

Q13. Why do LLMs hallucinate? A. They are trained to produce plausible continuations, not to check facts; missing, outdated or private knowledge gets filled with fluent guesses. Defences: grounding (retrieval, tools), validation, citations, evaluation, and letting the model say "I don't know". *(→ §4.10)*

Q14. What is the KV cache and what does it cost? A. Stored key and value vectors for earlier tokens, so each new token attends to them without recomputing. It saves compute but uses GPU memory that grows with context length × batch size — often the real limit on concurrency. *(→ §4.9, §11.4)*

Prompting and LLM apps (Chapters 5–6)

Q15. What goes into a good prompt? A. A clear task, relevant context, constraints (audience, length, tone), the output format, and examples when the format is subtle; delimiters to separate instructions from data; then iterate against test cases. *(→ §5.2)*

Q16. Zero-shot, few-shot, or chain-of-thought? A. Zero-shot for simple, well-known tasks; few-shot when format or style must be shown; step-by-step reasoning prompts (or a reasoning model) for multi-step logic and maths. *(→ §5.3, §5.7)*

Q17. Why is prompt injection structurally hard to stop? A. Your instructions and untrusted text arrive mixed together in one context window, with no hard wall the model is guaranteed to respect. Design as if it will sometimes be tricked: screen inputs, mark untrusted content, check outputs and actions, and keep tool permissions minimal so a tricked model has little it can break. *(→ §5.13, §12.4)*

Q18. How do you get reliable JSON out of a model? A. Use schema-constrained structured outputs or strict tool definitions where the API offers them; still validate the values; fall back to validate-and-retry. *(→ §6.4)*

Q19. Walk through the tool-use loop. A. Describe tools with JSON Schema → the model returns a tool call → your code validates the arguments, checks permissions and runs it → the result goes back to the model tied to the call's id → repeat until the model gives a final answer, with a step limit. *(→ §6.5)*

Q20. Which API errors do you retry, and how? A. Retry rate limits (429), server errors (5xx) and timeouts with exponential backoff plus jitter, honouring any retry-after header; do not retry other client errors (400 bad request, 401/403 auth) — fix the request instead; make side effects idempotent. *(→ §6.7)*

Q21. What is MCP and what problem does it solve? A. The Model Context Protocol: an open standard where a host application's clients talk JSON-RPC to servers exposing tools, resources and prompts. It turns M apps × N integrations into M + N; every server still runs with your permissions, so vet and scope it. *(→ §6.10)*

Retrieval, adaptation and evaluation (Chapters 7–8)

Q22. Describe a RAG pipeline. A. Offline: ingest documents → clean → chunk → embed → index (with metadata and permissions). Online: rewrite the query if needed → retrieve (hybrid) → rerank → assemble context → generate with citations → verify. *(→ §7.1, §7.6)*

Q23. Why combine keyword and vector retrieval rather than use vectors alone? A. Keyword search nails exact IDs and rare terms; vector search handles paraphrase and concepts. Real query mixes need both, fused (e.g. reciprocal rank fusion) and optionally reranked — evaluate on your own queries. *(→ §7.7)*

Q24. Where do most RAG failures come from? A. Don't assume one stage. Check separately: is the answer in the corpus (coverage)? was it retrieved (recall)? did it make it into the context? did the model use it faithfully? Each failure has a different fix. *(→ §7.11)*

Q25. In a RAG system, where and how are access rights enforced? A. Filter by the user's current access inside the retrieval query and re-check before anything reaches the model, caches or logs; fail closed. Filtering after generation is too late — the model has already seen it. *(→ §7.9, §7.12)*

Q26. Does RAG eliminate hallucination? A. No. Models can still contradict or embellish retrieved text. Defend in layers: answer-only-from-context instructions, citations checked to resolve, per-claim support checks for high-stakes answers, and refusal as a designed outcome. *(→ §7.8)*

Q27. RAG or fine-tuning? A. Ask what is failing. Missing or changing facts → RAG (updatable, citable, permission-aware). Wrong behaviour, format or narrow skill → prompting, then fine-tuning. Correct but too expensive → fine-tune a smaller model. *(→ §8.1)*

Q28. How do you evaluate outputs with no single right answer? A. Layer it: programmatic checks for what code can assert; an LLM judge with a rubric, validated against human labels; expert review for high stakes; a held-out golden set run on every change; online outcomes to confirm. *(→ §8.5)*

Q29. What biases do LLM judges have? A. Preference for longer answers, for the first-position option, and for their own model family's style; mitigate with swapped orderings, clear rubrics, calibration against humans, and a different judge model. *(→ §8.5)*

Agents (Chapter 9)

Q30. Agent or workflow? A. Ask whether the sequence of steps is knowable in advance. If it can be drawn as a flowchart today, build a workflow — cheaper, testable, readable. Use an agent loop only where the path depends on discoveries, and bound it (steps, cost, time, actions). *(→ §9.1)*

Q31. Why treat a model's tool call as a proposal rather than an order? A. Authorisation lives in the runtime, not the model: it validates arguments, uses user-scoped credentials, ranks tools by consequence, and puts approval gates on irreversible actions. A fooled model can only propose. *(→ §9.3, §9.4)*

Q32. When do multiple agents beat one? A. Three cases: a subtask benefits from its own clean, small context; independent pieces can run side by side; or a stage needs its own tools or model. Otherwise multi-agent just adds cost and a longer trace. *(→ §9.7)*

Q33. How does an agent earn more autonomy? A. One action tier at a time, up a four-rung ladder: suggest only (shadow) → human-approved execution → bounded autonomy inside caps → audited autonomy. Promotion needs representative tests, reliable reviewers and verified outcomes (not approval rate alone); a regression demotes the tier automatically; caps, alarms and a per-tool kill switch never switch off. *(→ §9.10)*

Generative models beyond text (Chapter 10)

Q34. How does a diffusion model generate an image? A. Training teaches a network to predict the noise added to images at many noise levels; generation starts from pure noise and removes it step by step, steered by a text embedding (with classifier-free guidance). Latent diffusion does this in a compressed latent space for speed. *(→ §10.4, §10.5)*

Q35. Why is text-to-image serving asynchronous? A. One image can occupy a GPU for many seconds, so requests become jobs in priority queues (which also drive pricing and autoscaling), served by long-lived batched GPU workers, with moderation on both the prompt and the output. *(→ §14.4)*

Serving and inference (Chapter 11)

Q36. Time-to-first-token vs total latency? A. TTFT covers network, queueing, preprocessing and prefill of the prompt; total latency adds decoding every output token and any checks. Shorter prompts and prefix caching help TTFT; shorter outputs and faster decoding help the rest; streaming helps perceived latency. *(→ §11.2, §11.3)*

Q37. What limits how many requests one GPU can serve? A. Memory first — weights plus a KV cache per active sequence; then compute, memory bandwidth and scheduling. Continuous batching and paged KV memory raise concurrency. *(→ §11.5, §11.10)*

Q38. In what order do you optimise inference cost? A. Avoid work (caching, routing to smaller models, trimming context) → run faster (batching, KV/prefix caching, speculative decoding) → shrink the model (quantisation, distillation) — measuring quality, latency and cost at each step. *(→ §11.13, §11.14, §11.7)*

Q39. How do you estimate the cost of an LLM feature? A. Requests per day × (input tokens × input price + output tokens × output price), adjusted for cache hits and routing; say the number out loud and compare it to the value or the human baseline. *(→ §11.15)*

Q40. API or self-host? A. Sustained token volume versus the cost of a GPU fleet at realistic utilisation, gated by data governance and team skills. Start on an API; revisit with measured traffic. *(→ §11.15, §11.18)*

Safety and governance (Chapter 12)

Q41. What guardrail layers would you put around an LLM product? A. Input checks (injection, abuse, PII), untrusted-data tagging when assembling context, output checks (policy, groundedness, PII), action gates with least privilege, and monitoring plus incident response. *(→ §12.6)*

Q42. How can an AI system be biased, and what do you do about it? A. Through skewed training data, labels, proxies for protected attributes, or feedback loops. Measure outcomes per group with fairness metrics, fix

data and thresholds, document decisions, and keep humans accountable for high-impact uses. *(→ §12.3)*

Q43. What privacy risks are specific to LLM apps? A. Sensitive data in prompts and logs, memorised training data, leakage across users via caches or retrieval, and third-party data retention. Minimise and redact inputs, scope retrieval by user, set retention, and check provider terms. *(→ §12.5)*

System design (Chapters 13–14; predictive systems in the ML Handbook)

Q44. Outline a framework for an AI system design interview. A. Pin down the problem → quantified success targets → where data and labels come from → the simplest model that could work → serving path with budgets → the feedback loop (logs, evals, retraining, rollout) → failure modes with alarms, and cost × volume. *(→ §13.4)*

Q45. How do you ship a new model or prompt safely? A. Offline eval against the current version → shadow → a small, fixed group of users on the new version (canary or A/B) with a pre-agreed metric that triggers rollback → gradual ramp; for agents, an autonomy ladder per action tier. *(→ §13.8)*

Q46. Why must training and serving be separate systems, and what is training-serving skew? A. Training favours throughput over history; serving favours low latency and availability. Skew is when features are computed differently in the two paths – prevent it by reusing the same feature code in both paths, joining historical data as of each event's time, logging the features actually served, and testing that both paths agree. *(→ ML Handbook ch. 16)*

Q47. Why is every large ranking system a funnel? A. Scoring millions of items with an expensive model is too slow, so stages shrink the set: cheap retrieval of hundreds of candidates → heavier ranking → business and diversity rules. Evaluate retrieval recall and ranking quality separately. *(→ ML Handbook ch. 15, ch. 17)*

Q48. What is position bias in search and feeds? A. Items shown higher get more clicks regardless of relevance, so raw clicks mix relevance with exposure. Correct with randomised data or exposure models such as propensity weighting, and evaluate the correction. *(→ ML Handbook ch. 17)*

Q49. Data drift vs concept drift? A. Data drift: the input distribution changes. Concept drift: the relationship between inputs and outcomes changes. Distribution monitors hint at the first; only (often delayed) outcome labels prove the second. A drift alarm triggers investigation, not automatic retraining. *(→ ML Handbook ch. 16)*

Q50. Your fraud model blocks a transaction. What data problem does that create? A. Selective labels: you never learn what would have happened if it had been approved. Use investigated cases and other permitted evidence; any exploration must stay within approved low-risk limits – never approve known high-risk activity just to get labels. *(→ ML Handbook ch. 18)*

Q51. Why keep a small exploration slice in a learning loop? A. The model decides what users see, so it shapes its own future training data. Controlled exploration shows outcomes outside its usual choices (record the selection probabilities); it reduces but does not remove bias, and must respect safety limits. *(→ ML Handbook ch. 16)*

Q52. Why do ad-click models need calibration, not just good ranking? A. Predicted click probabilities feed into auction ranking and expected value, so the numbers themselves must be right, not only their order. Check calibration per segment and validate any correction on held-out data. *(→ ML Handbook ch. 18)*

Scoring yourself

- **Pass mark:** you name the variable that drives the choice without a hint – for example how late labels arrive, whether tokens are mostly input or output, how risky an action is, what queries look like, or how much is at stake. A memorised answer collapses when the interviewer changes the scenario; one you can re-derive adapts.
- **40+ strong answers** suggests good recall; it does not prove readiness – also practise whole designs and follow-ups.
- **Below 30:** work through the decision cheat sheets, then the linked sections.

Drilling it with code

`code/drill.py` reads the questions straight from this README, draws a seeded random session of five, and shows how a **Leitner** spaced-repetition schedule spaces reviews: a correct answer promotes a card to a box reviewed less often (1 → 3 → 7 → 14 days); a miss sends it back to box 1, to be seen again after a day.

```python
# code/drill.py (core)
REVIEW_EVERY_DAYS = {1: 1, 2: 3, 3: 7, 4: 14}      # box -> days until next review

def load_bank(path):
    text = path.read_text(encoding="utf-8")
    pairs = re.findall(r"^\*\*Q\d+\.\s*(.+?)\*\*\s*\nA\.\s*(.+)$", text, flags=re.M)
    return [(q.strip(), a.strip()) for q, a in pairs]

def leitner(results, box=1):
    steps = []
    for ok in results:
        box = min(box + 1, 4) if ok else 1
        steps.append((box, REVIEW_EVERY_DAYS[box]))
    return steps
```

Output (abridged):

```
Loaded 52 questions from the README.

1. What is the KV cache and what does it cost?
   (30 s, out loud)  anchor: Stored key and value vectors for earlier tokens, so each new token attends to
them without recomputing.
...
5. BFS, DFS or A* — which and why?
   (30 s, out loud)  anchor: BFS finds the fewest-steps path but stores the whole frontier (O(b^d) memory); ...

Leitner schedule for one card answered right, right, wrong, right:
  review 1: box 2 -> next review in 3 day(s)
  review 2: box 3 -> next review in 7 day(s)
  review 3: box 1 -> next review in 1 day(s)
  review 4: box 2 -> next review in 3 day(s)
```

15.5 Future trends (as of 2026)

These are **directions of travel** visible today, not forecasts of specific products or dates. Revisit them each year; some will stall.

Where models are heading

Covered in more depth in §4.15 of Chapter 4: longer and more reliable multi-step reasoning; answers grounded in tools and cited sources; multimodal input and output as standard; more capable agents; interpretability research; small specialised models, including on-device ones; better data efficiency; and architectural efficiency (mixture-of-experts, quantisation, speculative decoding, hybrid attention/state-space designs).

Agents maturing

The practical trend is not "fully autonomous AI" but **bounded autonomy**: agents given narrow tool sets, spending limits, audit trails and approval gates, and promoted to more independence as evidence accumulates (Chapter 9). Open standards for connecting models to tools, such as MCP (§6.10), make it easier to plug agents into real systems — and make permission design more important.

AI converging with other technologies

Pairing	What AI adds	Realistic now	Still largely experimental
AI + IoT (internet-connected sensors and devices)	Learns from sensor streams to predict and adapt	Predictive maintenance (spotting unusual vibration before a machine fails); smart thermostats that learn schedules; wearables that flag abnormal heart rhythms	City-wide adaptive traffic, fully automated infrastructure repair scheduling
AI + AR/VR (digital overlays on the real world / fully virtual worlds)	Understands the scene, gestures and speech in real time	Live translation and captions, object recognition overlays, adaptive training simulations (surgery, aviation, factory work)	Seamless multilingual virtual workplaces with AI co-designers
AI + robotics	Perception, planning and adaptation for machines that act physically	Warehouse picking and sorting, collaborative robots ("cobots") working beside people, agricultural drones, sidewalk delivery pilots	General-purpose household or humanoid robots; fully automated logistics end to end
AI + quantum computing	–	Quantum computers today are small and error-prone; research focuses on simulating molecules and materials	Any practical speed-up for machine learning is **unproven**; quantum speed-ups exist only for specific problem types, not computing in general

Two accuracy notes. First, quantum computers are not "exponentially faster" in general – only for particular problems (for example factoring, via Shor's algorithm, and certain simulations). Second, the security story runs the other way from what is often claimed: a large quantum computer would break today's common public-key encryption, which is why standards bodies (NIST published its first **post-quantum cryptography** standards in 2024) are moving systems to new algorithms.

The future of work: augmentation first

- **Automation** fits repetitive, rule-like tasks: routine data checks, document sorting, parts of assembly. **Augmentation** fits most knowledge work: AI drafts, summarises, searches and flags, and a person decides. Examples: AI helps screen large applicant pools but people interview; clinicians use AI to support diagnosis while keeping responsibility for care.
- **Changing roles more than new titles.** The biggest shift is *AI fluency inside existing jobs* – engineers, analysts, support leads and writers directing AI well: clear instructions, the right context, checking outputs. Many forecasts expected "prompt engineer" to become a big job category; in practice it rarely stuck as a title of its own, because the skill turned out to be useful in nearly every role.
- **Roles that are growing**: AI/LLM engineers who build and evaluate AI features; people who **audit** AI for bias, privacy and compliance (AI ethics, governance and risk roles); **AI product and project managers** who connect technical teams with business goals, budgets and rollout; evaluation and red-teaming specialists.
- **Skills that hold value**: fundamentals (data, statistics, systems), evaluation and measurement, communication, domain knowledge, and ethical judgement.

Regulation and governance

Rules are arriving in stages, and the dates move. The EU AI Act (in force since August 2024) is the clearest example: its bans and general-purpose-model duties already apply, and in 2026 the Digital Omnibus on AI (Regulation (EU) 2026/1744, in force 27 July 2026) pushed the high-risk obligations back to 2 December 2027 for Annex III uses such as hiring and credit and 2 August 2028 for AI inside Annex I products, added bans on generating non-consensual intimate imagery or child sexual abuse material from 2 December 2026, and gave generative systems already on the market until 2 December 2026 to mark their output under Article 50. Organisations are also adopting risk-management frameworks such as the NIST AI RMF. Expect documentation, evaluation evidence and human oversight to become normal parts of shipping AI, and check the current text before quoting a date (§12.12).

Self-check

1. Is quantum computing a near-term way to speed up model training? → No; practical ML speed-ups are unproven and today's machines are small and noisy.
2. What is the most common way AI changes jobs today? → People in their current jobs learn to direct AI well; whole-job replacement and brand-new titles are the smaller effects.
3. What does "bounded autonomy" mean for agents? → Limited tools, budgets and approvals, with independence widened only on evidence.

15.6 A learning roadmap

How to use it

Move through the stages in order; each ends with a small project that proves you can do it. Skip a stage only if you can already build its project without notes.

Stage	Learn	Where	Prove it with
0. Python	Syntax, functions, data structures, OOP, typing, testing	Python Handbook (ch. 1–15)	A tested command-line tool
1. ML foundations	Learning types, maths intuition, NumPy/pandas/scikit-learn, evaluation	ML Handbook (ch. 1–9)	A classifier with a proper train/validation/test split and a written evaluation
2. Deep learning to transformers	Neural nets, attention, tokens, embeddings	ML Handbook ch. 10–12; Chapters 3–4 here	Train a tiny model in PyTorch; explain attention with worked numbers
3. Building with LLMs	Prompting, API calls, structured output, tools, MCP	Chapters 5–6	A small assistant that calls one tool and returns validated JSON
4. Grounding and evaluation	Embeddings, vector search, RAG, fine-tuning vs RAG, eval sets	Chapters 7–8	A RAG app over your own notes with a 30-question golden test set
5. Agents	Tool loops, planning, memory, guardrails, multi-agent trade-offs	Chapter 9	A bounded agent with step/cost limits and an approval gate
6. Beyond text; production	Diffusion and multimodal models; serving, latency, cost; safety and governance	Chapters 10–12	A cost/latency estimate and a threat model for your RAG app
7. Design and interviews	Design frameworks, case studies, this review	Chapters 13–15; ML Handbook ch. 15–18	Narrate two full designs aloud in 45 minutes each

Keeping it sharp

1. **Drill the question bank** (§15.4) in any interview week, following the three drilling rules at the top of this chapter.
2. **Narrate one full design per session** – for example the inference-cost exercise in Chapter 11 (drills arithmetic) and a case study in Chapter 14 (drills control and safety).
3. **Critique real systems** you meet at work: check the invariants first (permissions below the model, a feedback loop, a cost estimate, alarms for silent failure).
4. **Read primary sources** – model cards, official docs, papers – and date-check every claim (§1.9).

Where to go next

AI systems still stand on classic distributed-systems machinery. Companion notes in this repository:

- System Design Patterns – caching, sharding, queues, replication and end-to-end designs.

- Designing Data-Intensive Applications notes – storage, replication, partitioning, streams, consistency in depth.
- Microservices patterns – service boundaries, communication, resilience, data consistency.
- Low-Level Design Handbook – object-oriented design for the code inside those services.

Final thoughts

Three habits outlast every model release: **experiment** (build small prototypes; failures teach fastest), **collaborate** (technical and non-technical people together catch what either misses), and **stay grounded** (measure quality, count the cost, and keep the people affected by the system in view).

15.7 Glossary

Alphabetical. "Ch. N" links to the chapter of this book that explains the term; "ML N" links to the companion ML Handbook.

Term	Meaning	Where
A* search	Best-first search ordering nodes by f = g + h (cost so far + heuristic estimate); optimal with an admissible heuristic	Ch. 2
A/B test	Randomly splitting live traffic between two versions to compare a metric	Ch. 13
Admissible heuristic	A heuristic that never overestimates the true remaining cost	Ch. 2
Agent (AI agent)	A system that uses a model in a loop to choose and take actions through tools toward a goal	Ch. 9
Agentic AI	AI that plans and acts (calls tools, takes steps) rather than only producing a single output	Ch. 1
AGI (artificial general intelligence)	Hypothetical AI matching humans across most cognitive tasks; no agreed definition	Ch. 1
Alignment	Making a model's behaviour match human intentions and values	Ch. 12
Alpha-beta pruning	Minimax optimisation that skips branches which cannot change the decision	Ch. 2
Approximate nearest neighbour (ANN)	Search that finds almost-closest vectors much faster than exact search	Ch. 7
Artificial intelligence (AI)	Building machines that perceive, reason and act to achieve goals	Ch. 1
Attention	Mechanism that lets each token weigh and mix information from other tokens	Ch. 3
Autoregressive	Generating one token at a time, each conditioned on those before it	Ch. 4
Backoff (exponential)	Waiting progressively longer between retries, usually with random jitter	Ch. 6
Backpressure	Slowing or rejecting incoming work when a system is saturated	Ch. 11
Base model	A pretrained model before instruction or preference tuning	Ch. 4
Batch inference	Precomputing predictions ahead of time for later lookup	ML 16
Batching (continuous)	Adding and removing sequences from a GPU batch at every decoding step to keep the GPU busy	Ch. 11
Beam search	Decoding that keeps the k most probable partial sequences at each step	Ch. 4
Bias (fairness)	Systematically worse or unfair outcomes for some groups	Ch. 12
BM25	Classic keyword-ranking formula based on term frequency and document length	Ch. 7
BPE (byte-pair encoding)	Tokenizer that builds a vocabulary by repeatedly merging the most frequent symbol pairs	Ch. 3
Breadth-first search (BFS)	Uninformed search that expands the shallowest nodes first (FIFO queue)	Ch. 2
Canary release	Sending a small slice of traffic to a new version before full rollout	Ch. 13
Cascade (model)	Trying a cheap model first and escalating to a stronger one only when needed	Ch. 11
Chain-of-thought (CoT)	Prompting or training a model to write intermediate reasoning steps	Ch. 5
Chinchilla scaling	Finding that compute-optimal training uses about 20 tokens per parameter	Ch. 4
Chunking	Splitting documents into passages sized for embedding and retrieval	Ch. 7
Citation (grounded)	A pointer from a claim in the answer to the source passage supporting it	Ch. 7

Term	Meaning	Where
Classifier-free guidance	Diffusion technique that strengthens adherence to the prompt by mixing conditional and unconditional predictions	Ch. 10
Cold start	Lack of history for a new user or item, making personalisation hard	ML 13
Concept drift	Change in the relationship between inputs and the outcome	ML 16
Constitutional AI	Anthropic's method of training a model to follow a written set of principles, using AI feedback	Ch. 12
Constraint satisfaction problem (CSP)	Problem of assigning values to variables so that all constraints hold	Ch. 2
Context window	Maximum number of tokens a model can read (and write) in one request	Ch. 4
Cosine similarity	Similarity of two vectors measured by the angle between them	Ch. 3
Cross-encoder	Model that scores a query and a document together; accurate but slow, used for reranking	Ch. 7
Data drift	Change in the distribution of model inputs over time	ML 16
Decode (phase)	Generating output tokens one at a time after prefill	Ch. 11
Decoder-only transformer	Transformer that generates text left to right using causal masking (GPT, Claude, Llama)	Ch. 3
Deep learning	Machine learning with many-layer neural networks	Ch. 1
Delimiters	Markers (e.g. XML tags) that separate instructions from data in a prompt	Ch. 5
Depth-first search (DFS)	Uninformed search that expands the deepest node first (LIFO stack)	Ch. 2
Diffusion model	Generative model that learns to reverse a gradual noising process	Ch. 10
Distillation	Training a smaller "student" model to imitate a larger "teacher"	Ch. 11
DPO (direct preference optimization)	Preference tuning directly from pairs of preferred and rejected answers, without a separate reward model	Ch. 4
Embedding	Dense vector representing the meaning of a token, text or item	Ch. 3
Encoder	Transformer stack that reads the whole input bidirectionally (BERT)	Ch. 3
Encoder-decoder	Architecture where an encoder reads the input and a decoder generates the output (T5, Whisper)	Ch. 3
Evaluation set (golden set)	Fixed, representative test cases with expected outcomes, run on every change	Ch. 8
Expert system	Rule-based AI encoding specialist knowledge as if-then rules (1970s–80s)	Ch. 1
Exploration (vs exploitation)	Occasionally trying options the model would not choose, to learn about them	ML 16
Fail closed	If a safety or permission check cannot run, deny rather than allow	Ch. 13
Faithfulness	Whether every claim in an answer is supported by the retrieved context	Ch. 7
Feature store	System holding feature values: offline for training, online for low-latency serving	ML 16
Feedback loop	Logging predictions and outcomes to evaluate, retrain and improve the system	Ch. 13
Few-shot prompting	Including a handful of worked examples in the prompt	Ch. 5
Fine-tuning	Continuing training of a pretrained model on task-specific data	Ch. 8

Term	Meaning	Where
Foundation model	Large model pretrained on broad data and adapted to many tasks	Ch. 1
Frontier (search)	The set of discovered but not yet expanded nodes	Ch. 2
Function calling	See tool use	Ch. 6
Funnel (ranking)	Multi-stage pipeline: cheap retrieval of many candidates, then heavier ranking of few	ML 15
GAN (generative adversarial network)	Generator and discriminator trained against each other	Ch. 10
Generative AI	AI that creates new content (text, images, audio, code)	Ch. 1
Genetic algorithm	Local search that evolves a population with selection, crossover and mutation	Ch. 2
GPU memory sizing	Estimating memory for weights plus KV cache plus overhead	Ch. 11
Greedy decoding	Always choosing the single most probable next token	Ch. 4
Grounding	Tying model answers to supplied sources or tool results	Ch. 7
Grouped-query attention (GQA)	Attention variant where several query heads share one key/value head, shrinking the KV cache	Ch. 3
Guardrails	Checks around a model (input, output, action) that enforce policy	Ch. 12
Hallucination	Fluent but false or unsupported model output	Ch. 4
Heuristic	Estimate of the remaining cost to the goal, used to guide search	Ch. 2
Hill climbing	Local search that always moves to a better neighbour; can get stuck in local optima	Ch. 2
HNSW	Graph-based approximate nearest-neighbour index (Hierarchical Navigable Small World)	Ch. 7
Human-in-the-loop	A person reviews or approves model outputs or actions	Ch. 9
Hybrid search	Combining keyword and vector retrieval results	Ch. 7
HyDE	Retrieving with a hypothetical answer's embedding instead of the raw question	Ch. 7
Idempotency	Property that repeating an operation has the same effect as doing it once	Ch. 6
Inference	Running a trained model to get outputs	Ch. 11
Instruction tuning (SFT)	Supervised fine-tuning on instruction–response pairs to make a model follow requests	Ch. 4
Interpretability	Understanding why a model produced an output, or what its internals represent	Ch. 4
Iterative deepening	Repeated depth-limited DFS with growing limits; BFS's completeness with DFS's memory	Ch. 2
IVF index	ANN index that clusters vectors and searches only the nearest clusters (nprobe)	Ch. 7
Jailbreak	Prompt crafted to make a model ignore its safety rules	Ch. 12
JSON Schema	Standard for describing the shape of JSON data; used for tools and structured output	Ch. 6
Knowledge cutoff	Date after which a model's training data has no information	Ch. 4

Term	Meaning	Where
KV cache	Stored attention keys and values for earlier tokens, reused during decoding	Ch. 11
Large language model (LLM)	Transformer trained on huge text corpora to predict the next token	Ch. 4
Latency budget	Maximum time allotted to each stage of a request	Ch. 13
Latent diffusion	Diffusion performed in a compressed latent space (Stable Diffusion)	Ch. 10
Layer normalization	Rescaling activations within a layer to stabilise training	Ch. 3
Least privilege	Granting only the minimum permissions a component needs	Ch. 9
LLM-as-a-judge	Using a model with a rubric to grade other model outputs	Ch. 8
LLMOps	Practices for versioning, evaluating, deploying and monitoring LLM features	Ch. 13
Local search	Optimisation that improves a single current state step by step	Ch. 2
Logits	Raw, unnormalised scores a model outputs before softmax	Ch. 4
LoRA	Low-rank adaptation: fine-tuning by training small added matrices instead of all weights	Ch. 8
Machine learning (ML)	Systems that learn patterns from data rather than following hand-written rules	Ch. 1
MCP (Model Context Protocol)	Open protocol connecting AI hosts to servers exposing tools, resources and prompts	Ch. 6
Memory (agent/conversation)	Stored information carried across turns or sessions within a context budget	Ch. 6
Minimax	Game search assuming both players play optimally	Ch. 2
Mixture of experts (MoE)	Layer where a router sends each token to a few of many expert sub-networks	Ch. 3
Model card	Document describing a model's intended use, data, evaluation and limits	Ch. 12
Monte Carlo tree search (MCTS)	Game search that grows a tree using random playouts and UCB1 selection	Ch. 2
MRV heuristic	Minimum remaining values: in a CSP, assign the variable with the fewest legal values first	Ch. 2
Multi-agent system	Several cooperating agents, each with its own context, tools or role	Ch. 9
Multi-head attention	Several attention heads in parallel, each learning different relations	Ch. 3
Multimodal model	Model that handles more than one data type (text, image, audio, video)	Ch. 4
Narrow AI	AI built for specific tasks	Ch. 1
Next-token prediction	Training objective: predict the following token given all previous ones	Ch. 4
Observability	Logs, metrics and traces that show what a system is doing and why	Ch. 11
Online inference	Computing predictions at request time	ML 16
Open-weight model	Model whose trained weights are published for download (Llama, Qwen, Mistral)	Ch. 4
Overfitting	Model memorises training data and fails on new data	Ch. 8
PagedAttention	KV-cache memory managed in fixed-size pages to cut fragmentation (vLLM)	Ch. 11
PEAS	Performance, Environment, Actuators, Sensors: a way to specify an agent's task	Ch. 1

Term	Meaning	Where
PEFT	Parameter-efficient fine-tuning: adapting a model by training a small fraction of parameters	Ch. 8
Perplexity	Exponential of average negative log-likelihood; lower = model less "surprised"	Ch. 4
PII	Personally identifiable information	Ch. 12
Pipeline parallelism	Placing different layers of a model on different GPUs or nodes	Ch. 11
Position bias	Higher-ranked items get more attention and clicks regardless of relevance	ML 17
Positional encoding	Information added so a transformer knows token order (sinusoidal, learned, RoPE)	Ch. 3
Prefill	Processing the whole prompt in parallel before generation starts	Ch. 11
Pretraining	Initial large-scale training on general data	Ch. 4
Product quantization (PQ)	Compressing vectors by encoding sub-vectors as short codes	Ch. 7
Prompt	The input instructions and context given to a model	Ch. 5
Prompt caching	Reusing the computed state of a repeated prompt prefix to save cost and latency	Ch. 6
Prompt chaining	Splitting a task into several prompts where each output feeds the next	Ch. 5
Prompt engineering	Designing and testing prompts to get reliable model behaviour	Ch. 5
Prompt injection	Untrusted content containing instructions that hijack the model	Ch. 12
Prompt registry	Versioned store of prompts used by an application	Ch. 13
Quantization	Storing weights or activations with fewer bits (e.g. 8- or 4-bit) to save memory and time	Ch. 11
RAG (retrieval-augmented generation)	Retrieving relevant documents and adding them to the prompt before generating	Ch. 7
Rate limit	Cap on requests or tokens per unit time	Ch. 6
Rational agent	An agent that acts to maximise its expected performance measure	Ch. 1
ReAct	Pattern interleaving reasoning steps with tool actions and observations	Ch. 9
Reasoning model	LLM trained (often with RL) to think in extended intermediate steps before answering	Ch. 4
Recall@k	Share of relevant items that appear in the top k results	Ch. 7
Reciprocal rank fusion (RRF)	Merging ranked lists by summing 1/(k + rank) for each item	Ch. 7
Red teaming	Deliberately attacking a system to find failures before users do	Ch. 12
Reinforcement learning (RL)	Learning by trial and error from rewards	Ch. 4
Reranker	Second-stage model that reorders retrieved candidates more precisely	Ch. 7
Residual connection	Adding a layer's input to its output, easing training of deep stacks	Ch. 3
Reward model	Model trained on human preference data to score responses, used in RLHF	Ch. 4

Term	Meaning	Where
RLHF	Reinforcement learning from human feedback: reward model trained on preferences, then policy optimisation	Ch. 4
RoPE	Rotary positional embedding: rotates query/key vectors by position	Ch. 3
Routing (model)	Picking, per request, the least expensive model able to do the job well	Ch. 11
Scaling laws	Power-law relationships between loss and model size, data and compute	Ch. 4
Scenario-based prompting	Asking the model to play a stakeholder ("act as a recruiter") to get feedback from that viewpoint	Ch. 5
Self-attention	Attention where queries, keys and values all come from the same sequence	Ch. 3
Self-consistency	Sampling several reasoning paths and taking the majority answer	Ch. 5
Semantic cache	Cache that returns a stored answer for a query similar in meaning	Ch. 11
Shadow deployment	Running a new version on live traffic without showing its outputs to users	Ch. 13
Simulated annealing	Local search that sometimes accepts worse moves, less often as "temperature" falls	Ch. 2
Softmax	Function turning scores into probabilities that sum to 1	Ch. 3
Speculative decoding	A small draft model proposes tokens that the large model verifies in one pass	Ch. 11
Stop reason	Why the model stopped generating (end of turn, max tokens, tool use, stop sequence)	Ch. 6
Streaming	Sending output tokens to the client as they are generated	Ch. 6
Structured output	Model output constrained to a schema, such as JSON	Ch. 6
Symbolic AI	AI built on explicit symbols, logic and hand-written rules	Ch. 1
Synthetic data	Training or evaluation data generated by models or programs	Ch. 8
System prompt	High-priority instructions setting a model's role and rules for a conversation	Ch. 5
Temperature	Sampling setting that sharpens (low) or flattens (high) the next-token distribution	Ch. 4
Tensor parallelism	Splitting each layer's matrices across several GPUs	Ch. 11
Throughput	Amount of work (e.g. tokens per second) a system completes per unit time	Ch. 11
Time to first token (TTFT)	Delay from sending a request until the first output token arrives	Ch. 11
Token	Unit of text a model processes, often a subword	Ch. 3
Tokenizer	Component that splits text into tokens and maps them to IDs	Ch. 3
Tombstone	Marker recording that an item was deleted, so caches and indexes drop it too	Ch. 7
Tool use	Model requests to call external functions, which your code executes and returns	Ch. 6
Top-k sampling	Sampling only from the k most likely next tokens	Ch. 4
Top-p (nucleus) sampling	Sampling from the smallest set of tokens whose probabilities sum to p	Ch. 4

Term	Meaning	Where
TPOT	Time per output token during decoding	Ch. 11
Training-serving skew	Features computed differently in training and in production	ML 16
Transformer	Neural architecture built from attention and feed-forward blocks (2017)	Ch. 3
Transition model	Description of the state that results from taking an action in a state	Ch. 2
Turing test	Imitation game: can a machine pass as human in text conversation?	Ch. 1
Two-tower model	Separate encoders for users/queries and items whose embeddings are compared for retrieval	ML 13
UCB1	Formula balancing exploration and exploitation when choosing which move to simulate in MCTS	Ch. 2
Uniform-cost search	Expands the node with the lowest path cost g (Dijkstra's algorithm on graphs)	Ch. 2
VAE (variational autoencoder)	Generative model that encodes data into a probabilistic latent space and decodes samples	Ch. 10
Vector database	Store and index for embeddings supporting fast nearest-neighbour search	Ch. 7
Workflow (vs agent)	Fixed, pre-designed sequence of LLM and code steps	Ch. 9
Zero-shot prompting	Asking a model to do a task with instructions but no examples	Ch. 5

Verifying the cross-references

Every bullet in this chapter points to a section elsewhere, so the links were checked mechanically. `code/check_refs.py` scans each chapter README for relative links, confirms the target file exists, and — when the link text names a section such as "§7.9" — confirms the target really has a `## 7.9` heading. Run it after editing any chapter.

```
# code/check_refs.py (core)
LINK = re.compile(r"(!?)\[([^\]]*)\]\(([^)\s]+)\)")
SECTION = re.compile(r"§\s?(\d+\.\d+)")

for bang, label, target in LINK.findall(line):
    path = (readme.parent / target.split("#")[0]).resolve()
    if not path.exists():
        problems.append(f"missing {target}")
    for sec in SECTION.findall(label):          # "§7.9" must be a real "## 7.9" heading
        if sec not in headings(path):
            problems.append(f"§{sec} not in {target}")
```

Output:

```
Checked 16 README file(s): 0 problem(s).
```

Chapter TL;DR

- **Revise with rhythm**: review sheets per chapter, decision sheets daily, red flags once plus the night before, the question bank shuffled and aloud, one full design per session; re-drill misses after a day or two.
- **The sentence to master**: "The deciding variable here is X; ours is Y, so Z — at the cost of W."
- **Absolutes**: permissions enforced below the model and failing closed; every design closes the loop (logs → evals → retraining → safe rollout); every design says its cost (unit cost × volume) and its silent failures with alarms.

- **Most common red flags**: no numbers, no feedback loop, no cost estimate — all cheap to fix.
- **Trends (2026)**: stronger reasoning, grounded and multimodal models, bounded agents, small on-device models, AI combined with IoT, AR/VR and robotics; quantum speed-ups for ML unproven; work is mostly augmented rather than replaced; regulation phasing in.
- **Roadmap**: Python → ML foundations → deep learning and transformers → building with LLMs → grounding and evaluation → agents → production and safety → design and interviews, each stage proved by a project.
- **Glossary**: 183 terms, each linked to the chapter that teaches it.